

B+树

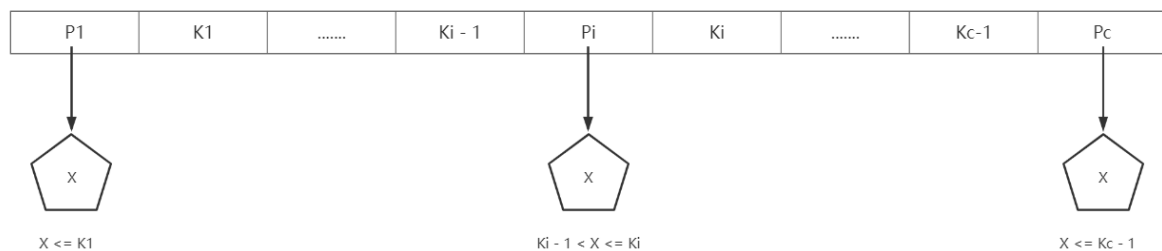
概念

为了实现动态多层索引，通常采用 B-树 和 B+树。但是，用于索引的 B-树 存在缺陷，它的所有中间结点均存储的是数据指针（指向包含键值的磁盘文件块的指针），与该键值一起存储在 B-树的结点中。这就会导致可以存储在 B-树中的结点数极大地减少了，从而增加 B-树的层数，进而增加了记录的搜索时间。

B+树通过仅在树的叶子结点中存储数据指针而消除了上述缺陷。因此，B+树的叶结点的结构与 B-树的内部结点的结构完全不同。在这里应该注意，由于数据指针仅存在于叶子结点中，因此叶子结点必须将所有键值及其对应的数据指针存储到磁盘文件块以便访问。此外，叶子结点被链接磁盘的某个位置，以提供对记录的有序访问。因此，叶子结点形成第一级索引，而内部结点形成多层索引的其他层。叶子结点的某些关键字 key 也出现在内部结点中，充当控制搜索记录的媒介。

阶为a的B+树内部结点的结构

与 B-树不同，B+树中的结点存在两个阶（order）：对于阶“a”和“b”，一个用于内部结点，另一个用于外部（或叶）结点。



1、对于每一个类似于

$$\langle P_1, K_1, P_2, K_2, \dots, P_{c-1}, K_{c-1}, P_c \rangle$$

的内部结点，其中 $c \leq a$, 每一个 P_i 表示指向子树根节点的指针， K_i 表示关键字。

2、对于每一个内部结点的关键字，均满足

$$K_1 < K_2 < \dots < K_{c-1}$$

内部结点的关键字从小到大排列。

3、对于一个位于 P_i 所指向的子树中的 X 而言，满足

当 $1 < i < c$ 时，均有 $K(i-1) < X \leq K_i$

当 $i = c$ 时， $X > K(c-1)$

当 $i = 1$ 时， $X \leq K_1$

4、每一个内部结点最多有 a 个指向子树的指针，即 c 最大取 a 。

5、根节点至少包含两个指向子树的结点指针，即对于根节点而言 $2 \leq c \leq a$ ；除了根之外的每个结点都包含最少 $\text{ceil}(a/2)$ 个指向子树的指针。

6、如果任意一个内部结点包含 c 个指向孩子结点的指针且 $c \leq a$ ，则该结点包含 $c - 1$ 的关键字。

阶为b的B+树叶子结点的结构



1、对于每一个形如

$$\langle \langle K_1, D_1 \rangle, \langle K_2, D_2 \rangle, \dots, \langle K_{c-1}, D_{c-1} \rangle, P_{next} \rangle$$

的叶子结点，其中 $c \leq b$ ， D_i 是一个数据指针，主要是指向磁盘上的值等于 K_i 的真实记录的指针，或者说包含 K_i 的磁盘文件块。 K_i 是一个关键字， P_{next} 表示B+树中指向下一个叶子结点的指针。

2、对于任意一个叶子结点均有：

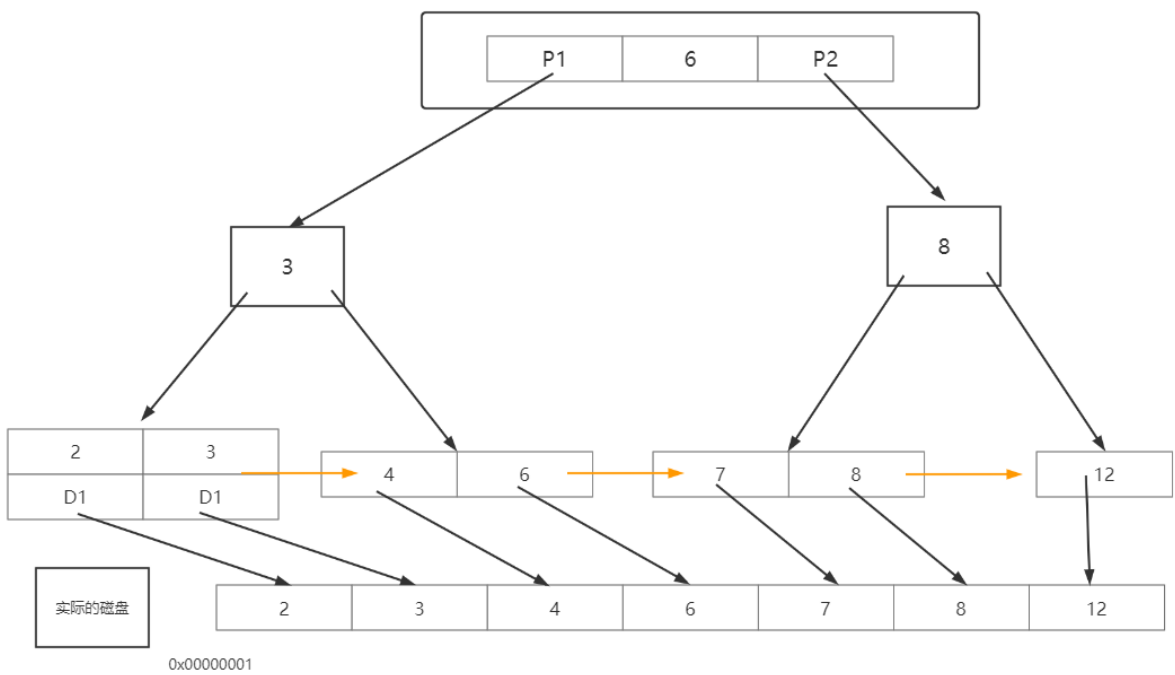
$$K_1 < K_2 < \dots < K_{c-1}$$

3、每一个叶子结点至少包含 $\text{ceil}(a/2)$ 的值

4、所有的叶子结点在同一层

使用 P_{next} 就可以遍历所有的叶子结点，就和单链表一样，从而实现对磁盘上记录的有序访问

一颗B+树的结构示意图

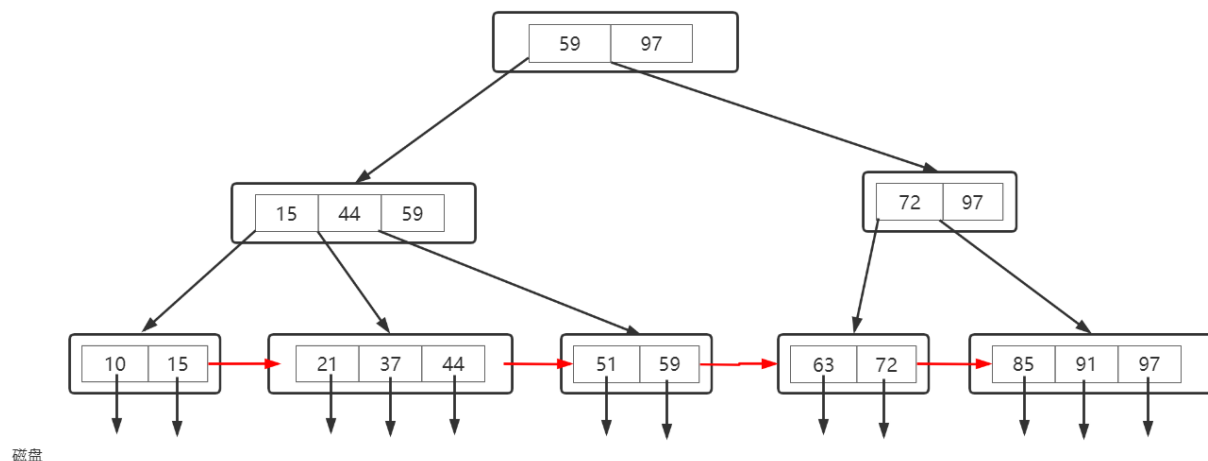


B+树的优点

同为h层的 B-树和 B+树，B+树可以存储更多的结点元素，更加“矮胖”。这也是 B+树最大的优势所在，极大地改善了 B-树的查找效率。对于同样多的记录，B+树的高度会更矮，并且Pnext指针的出现可以帮助 B+树快速访问磁盘记录且效率非常高。总之，就是 B+树比 B-树更加好，B+树的磁盘 I/O 会更少，相比于 B-树的中序遍历，B+树只需要像遍历单链表一样扫描一遍叶子结点。

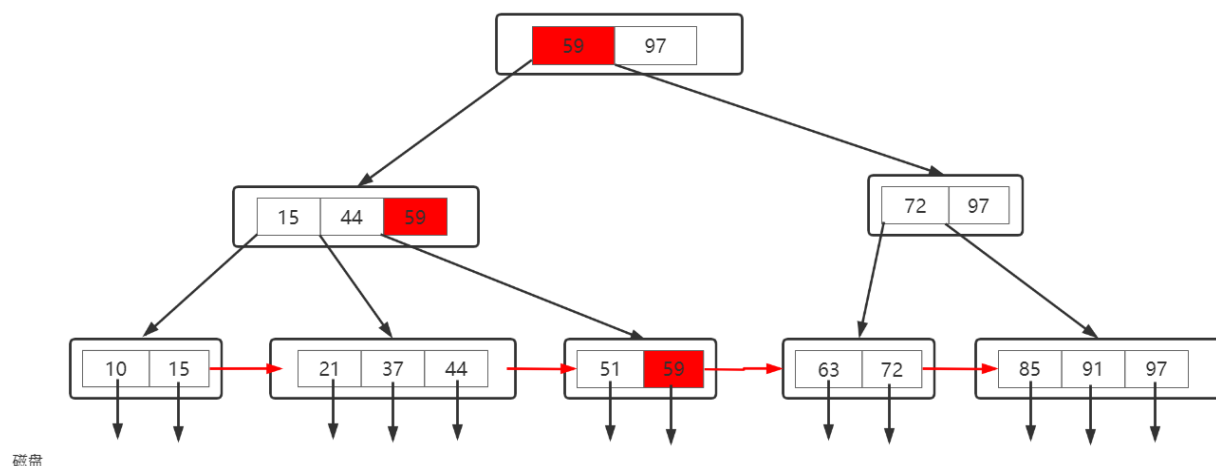
B+树的查找

前面谈了B+树的基本概念，我们就再来看一下B+树的查找操作。



下面所有的查找操作都是在这颗 B+ 树上进行了，为此，我们先仔细观察一下这颗 B+ 树（毫不隐瞒，这颗 B+ 树出自于严蔚敏老师的数据结构教材）。

第一点：B+ 树中的所有数据均保存在叶子结点，且根结点和内部结点均只是充当控制查找记录的媒介，并不代表数据本身，所有的内部结点元素都同时存在于子结点中，是子节点元素中是最大（或最小）元素。



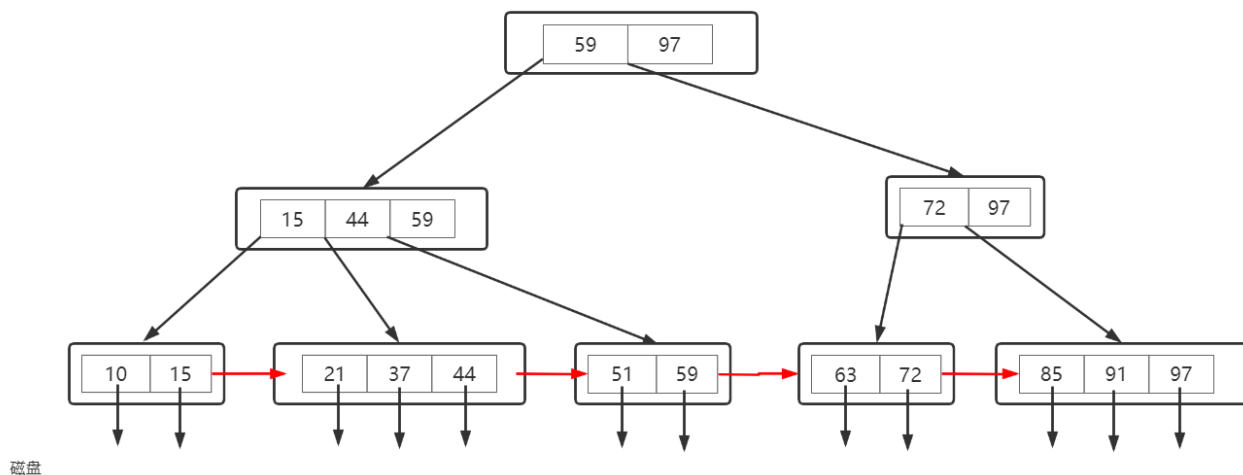
比如 B+ 树中的结点 59（结点 15、44、97、72 类似），是其子结点 [15、44、59] 中的最大元素，也是叶子结点 [51、59] 中的最大元素。所有的数据 [10、15、21、37、44、51、59、63、72、85、91、97] 均保存在叶子结点之中，而根结点 [59、97] 及内部结点 [15、44、59] 与 [72、97] 均不是数据本身，只是充当控制查找记录的媒介。

需要注意的是，根结点的最大元素 97 是整颗 B+ 树当中最大的元素，无论之后在叶子结点中插入或删除多少元素，始终要保证最大元素在根结点当中，这个讲插入和删除时还会看到。

第二点：每一个叶子结点都有指向下一个叶子结点的Pnext指针，便捷之处就在于之后我们将看到的区间查找。

单个查询元素

我们查找59为例进行说明。

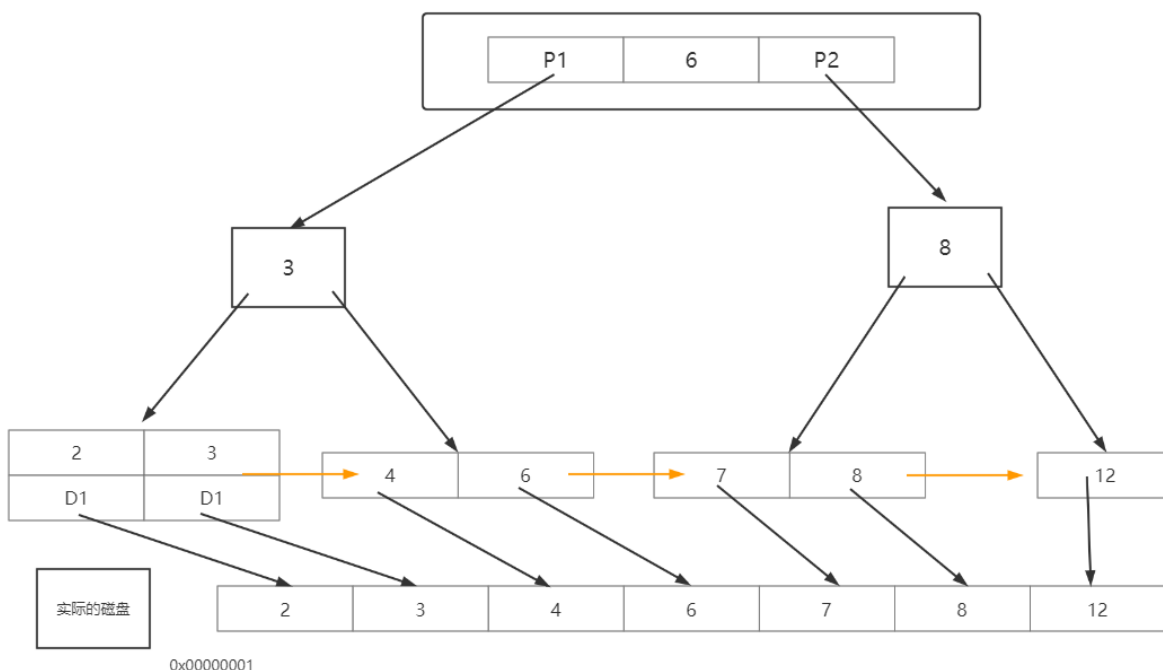


第一次磁盘 I/O：访问根结点 $[59, 97]$ ，发现 59 小于等于 $[59, 97]$ 中的 59，则访问根结点的第一个孩子结点。

第二次磁盘 I/O：访问结点 $[15, 44, 59]$ ，发现 59 大于 44 且小于等于 59，则访问当前结点的第三个孩子结点 $[51, 59]$ 。

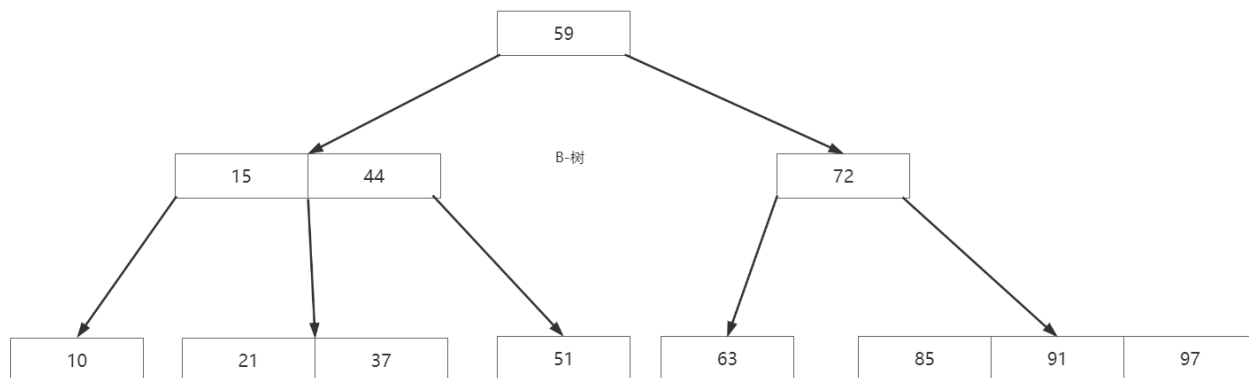
第三次磁盘 I/O：访问叶子结点 $[51, 59]$ ，顺序遍历结点内部，找到要查找的元素 59。

我想你已经注意到了和 B-树的区别，对于 B+树中单个元素的查找而言，每一个元素都有相同的磁盘 I/O 操作次数，即使查询的元素出现在根结点中，但那只是一个充当控制查找记录的媒介，并不是数据本身，数据真正存在于叶子结点当中，所以 B+树中查找任何一个元素都要从根结点一直走到叶子结点才可以。



B+树的非叶子结点均不存储 Data (即 D_i ，官方将其称为卫星数据)，所以与 B-树相比，同样大小的磁盘页，B+树的非叶子结点可以存储更多的索引（关键字），这也就意味着在数据量相同的情况下，B+树的结构比 B-树更加“矮胖”，查询时磁盘 I/O 次数会更少。

注意：B-树的查询性能并不稳定，对于根结点中关键字可能只有一次磁盘 I/O，而对于叶子结点中的关键字需要树的高度次磁盘 I/O 操作。

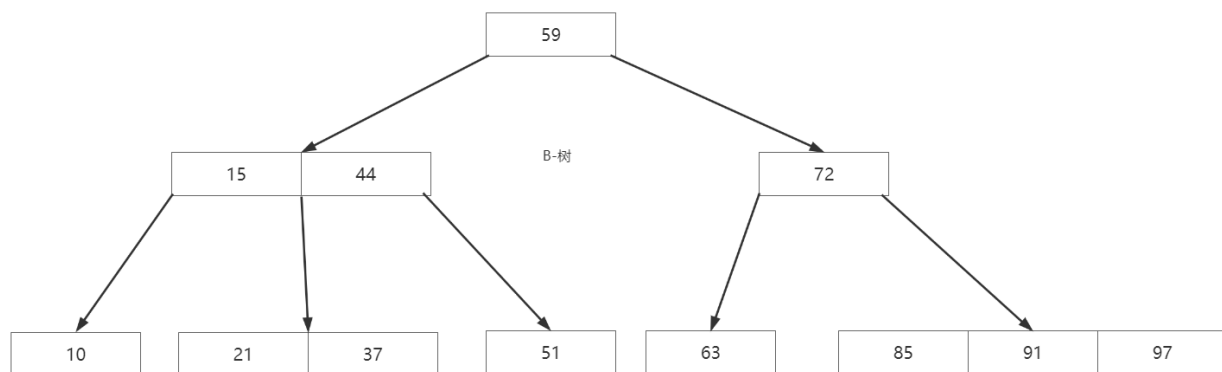


比如查找上图 B-树中的关键字 59 仅需要一次磁盘 I/O 操作，关键字 21 需要 3 次磁盘 I/O，关键字 72 需要 2 次磁盘 I/O。

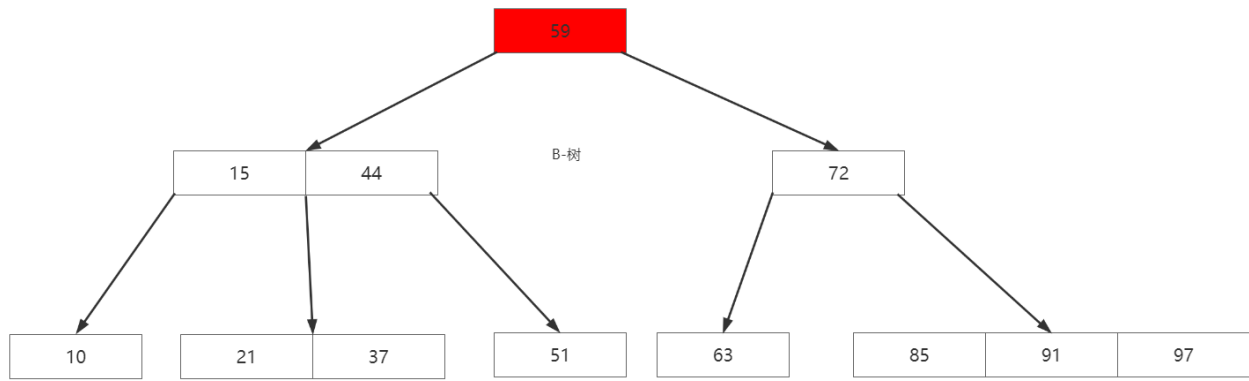
B+树所有查询所有关键字的磁盘 I/O 的次数都是树的高度。

区间查询

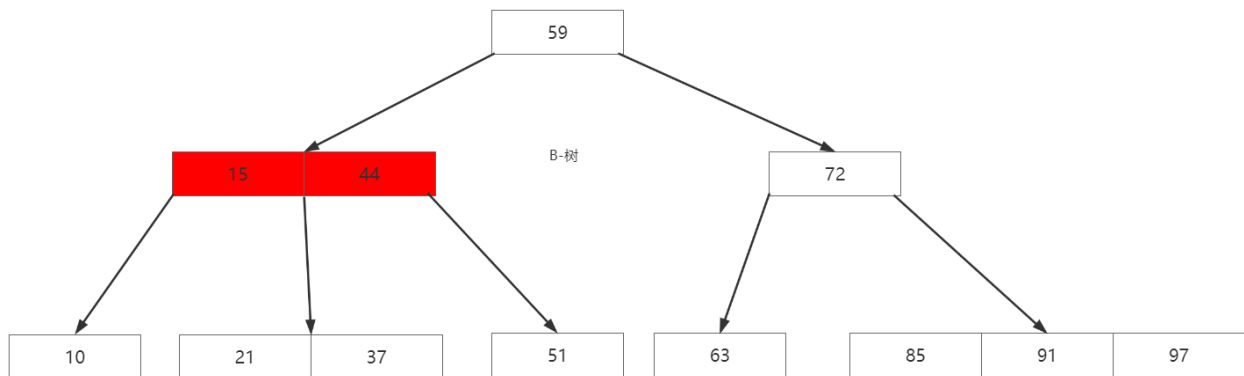
为了更清楚地看到 B+树进行区间查询的优势，我们以查询下面的 B-树中大于等于 21，小于等于 63 的关键字为例进行说明。



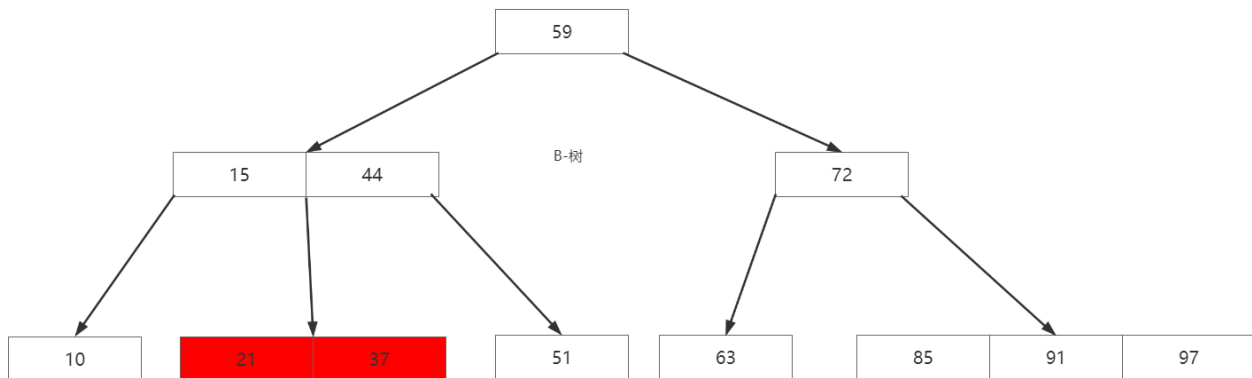
第一步：访问 B-树的根结点 [59]，发现 21 比 59 小，则访问根结点的第一个孩子 [15、44]。



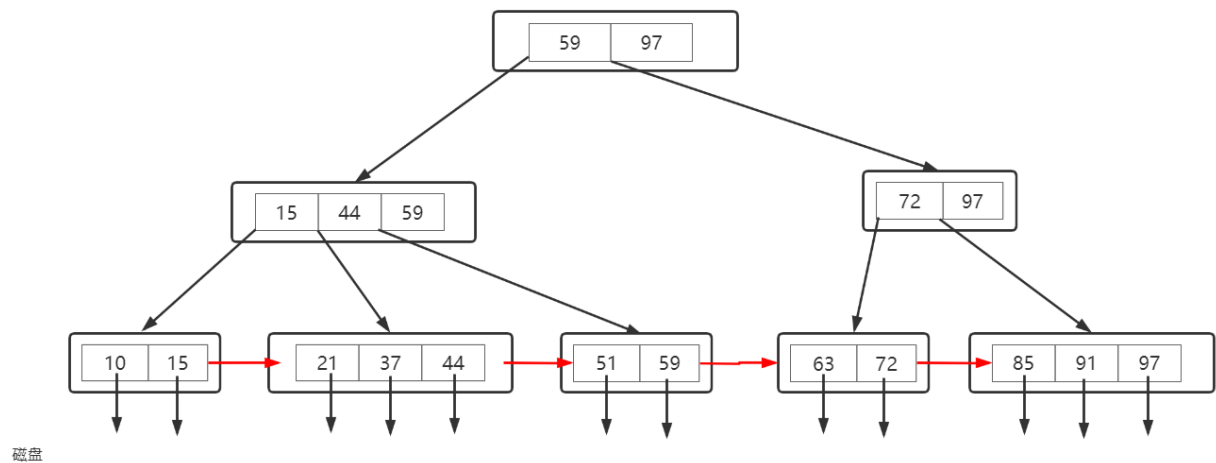
第二步：访问结点 $[15, 44]$ ，发现 21 大于 15 且小于 44，则访问当前结点的第二个孩子结点 $[21, 37]$ 。



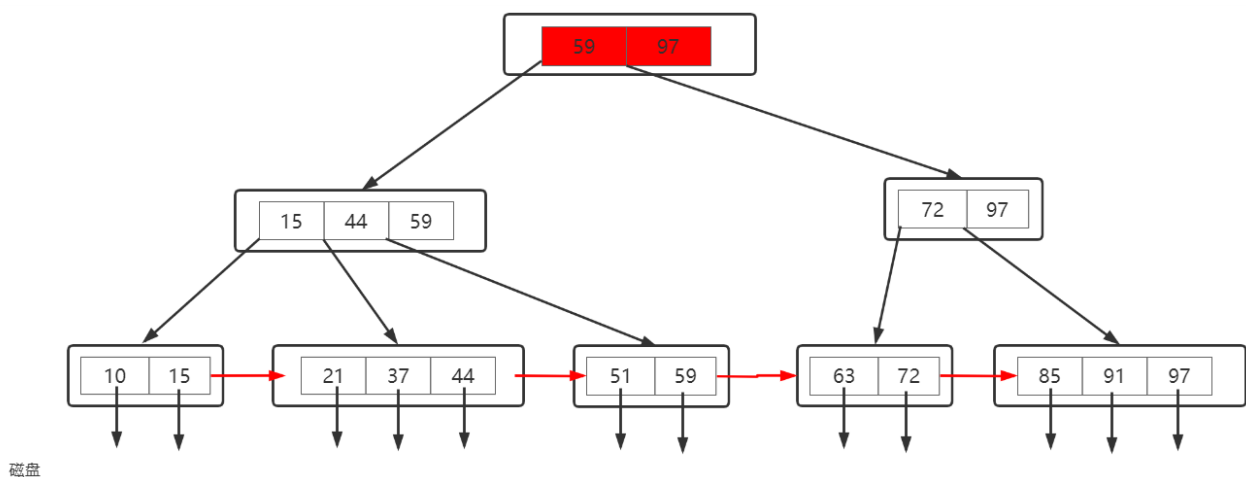
第三步：访问结点 $[21, 37]$ ，找到区间的左端点 21，然后从该关键字 21 开始，进行中序遍历，依次为关键字 37、44、51、59，直到遍历到区间的右端点 63 为止，不考虑中序遍历过程的压栈和入栈操作，磁盘 I/O 次数多了 2 次，即访问结点 72 和结点 63 并加载进内存。



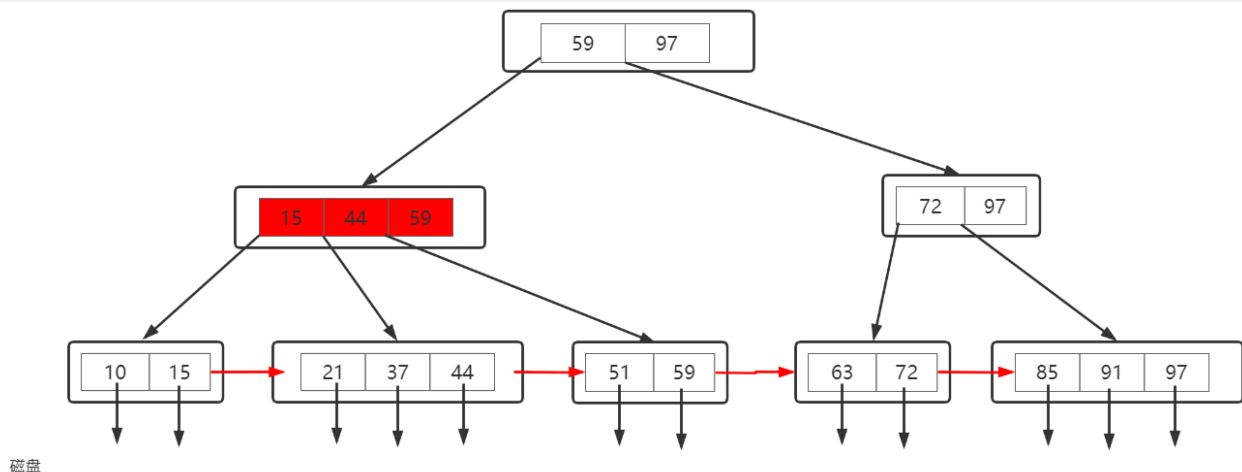
而 B+树进行区间查找，简直要舒服的不要不要的。同样是查找区间 $[21, 63]$ 之间的关键字。



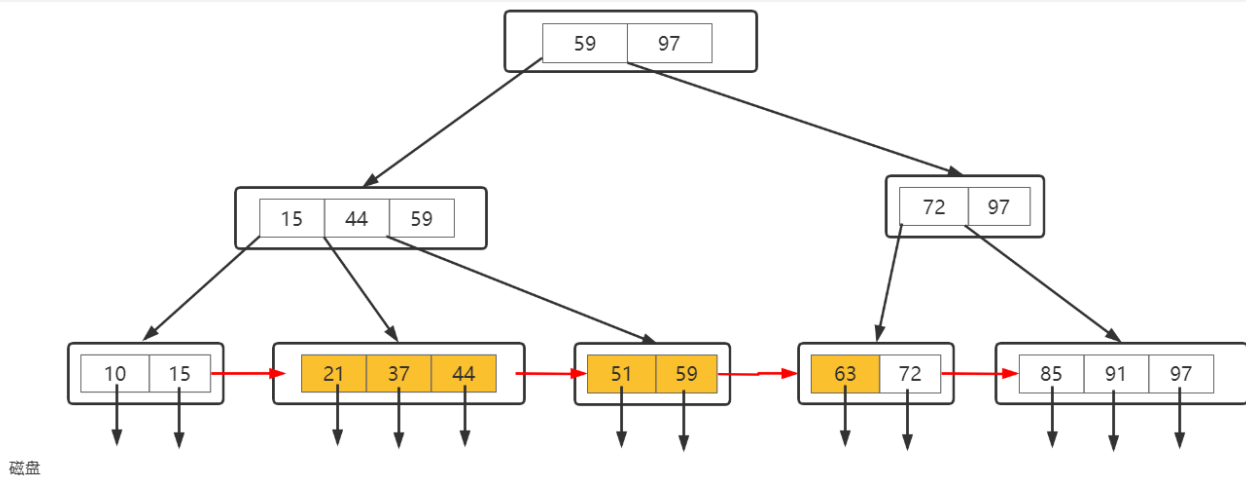
第一步：访问根结点 $[59, 97]$ ，发现区间的左端点 21 小于 59，则访问第一个孩子结点 $[15, 44, 59]$ 。



第二步：访问结点 $[15, 44, 59]$ ，发现 21 大于 15 且小于 44，则访问第二个孩子结点 $[21, 37, 44]$ 。



第三步：访问结点 $[21, 37, 44]$ ，找到了左端点 21，此时 B+树的优越性就出来了，不再需要中序遍历，而是相当于单链表的遍历，直接从左端点 21 开始一直遍历到左端点 63 即可，没有任何额外的磁盘 I/O 操作。



综合来看 B+树的优势就是：

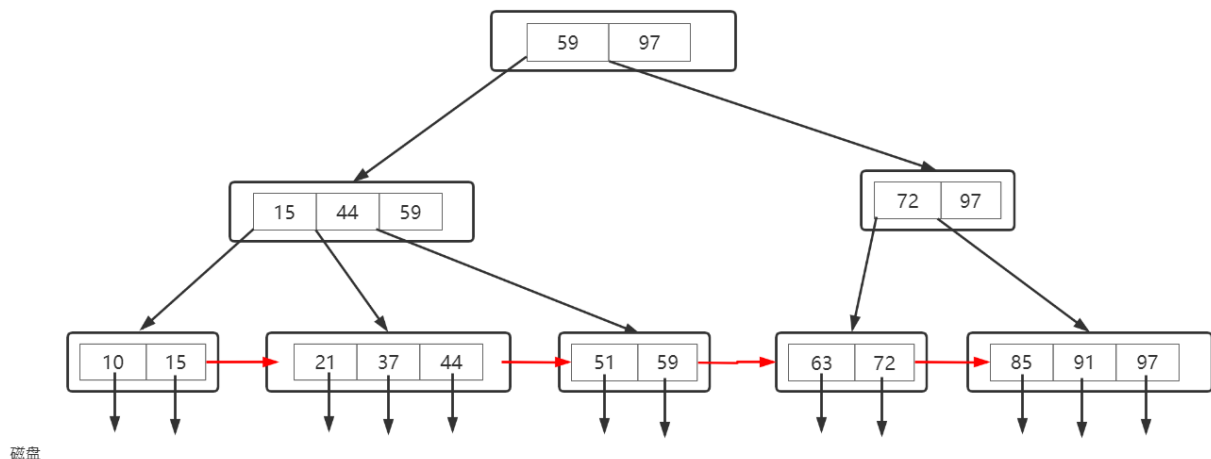
1. 查找时磁盘 I/O 次数更少，因为 B+树的非叶子结点可以存储更多的关键字，数据量相同的情况下，B+树更加“矮胖”，效率更高。
2. B+树查询所有关键字的磁盘 I/O 次数都一样，查询效率稳定。
3. B+树进行区间查找时更加简便实用。

B+树的插入操作

在B+树中插入关键字时，需要注意以下几点：

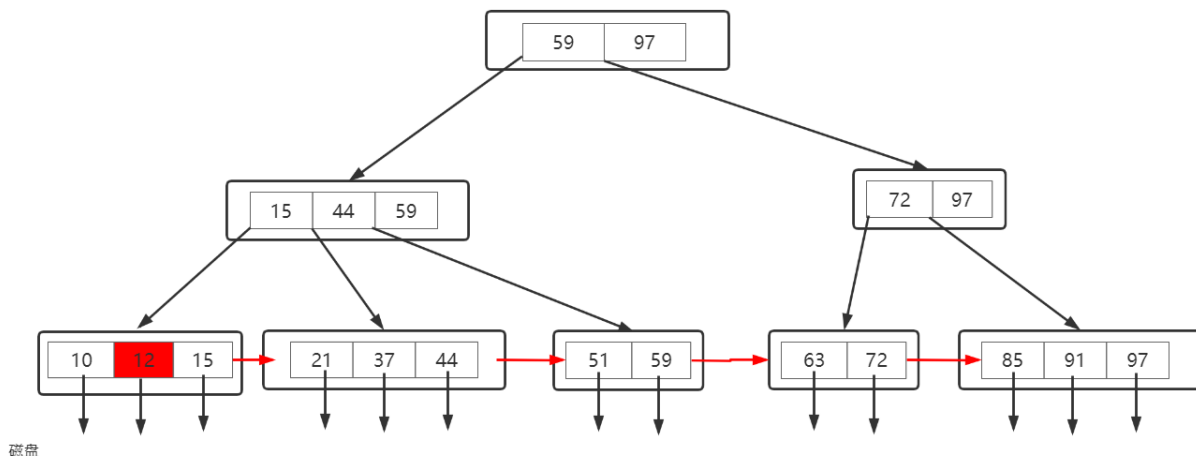
- 插入的操作全部都在叶子结点上进行，且不能破坏关键字自小而大的顺序；
- 由于 B+树中各结点中存储的关键字的个数有明确的范围，做插入操作可能会出现结点中关键字个数超过阶数的情况，此时需要将该结点进行“分裂”；

我们依旧以之前介绍查找操作时使用的图对插入操作进行说明，需要注意的是，B+树的阶数 $M = 3$ ，且 $\lceil M/2 \rceil = 2$ （取上限）、 $\lfloor M/2 \rfloor = 1$ （取下限）：



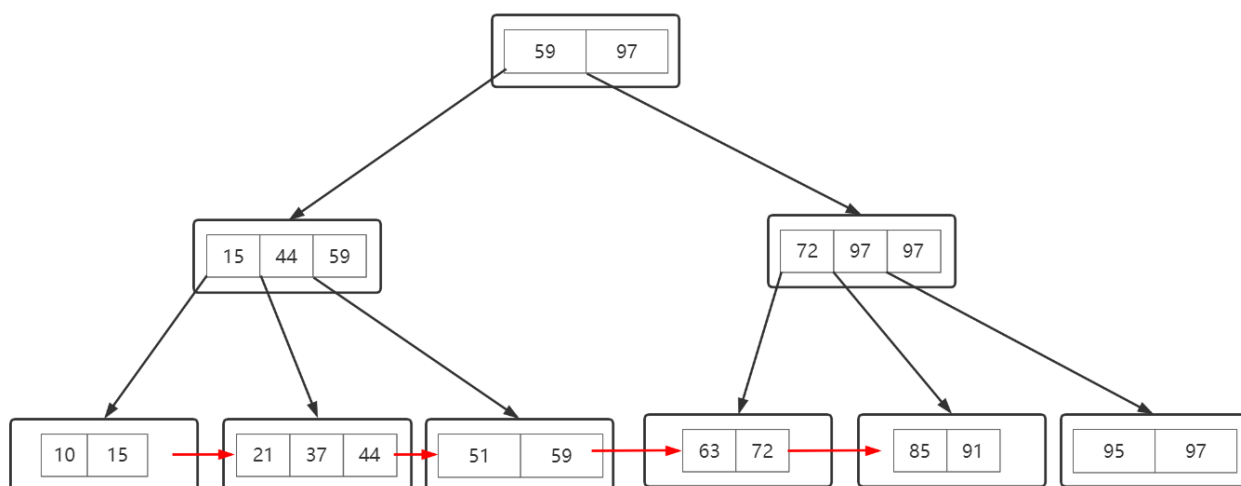
B+树中做插入关键字的操作，有以下 3 种情况：

- 1、若被插入关键字所在的结点，其含有关键字数目小于阶数 M ，则直接插入；比如插入关键字 12，插入关键字所在的结点的 $[10, 15]$ 包含两个关键字，小于 M ，则直接插入关键字 12。



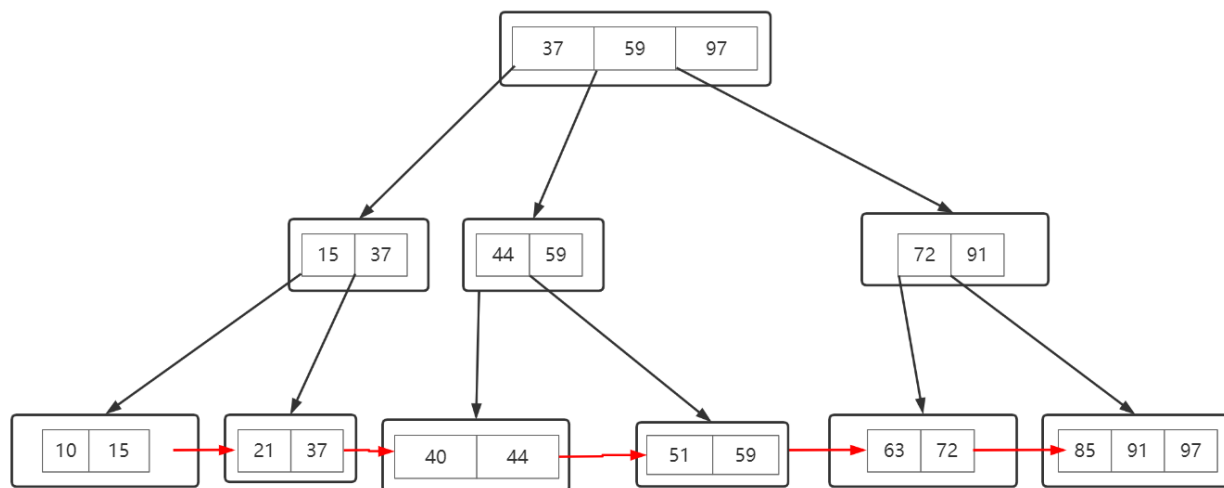
2、若被插入关键字所在的结点，其含有关键字数目等于阶数 M ，则需要将该结点分裂为两个结点，一个结点包含 $\lfloor M/2 \rfloor$ ，另一个结点包含 $\lceil M/2 \rceil$ 。同时，将 $\lceil M/2 \rceil$ 的关键字上移至其双亲结点。假设其双亲结点中包含的关键字个数小于 M ，则插入操作完成。

插入关键字 95，插入关键字所在结点 $[85, 91, 97]$ 包含 3 个关键字，等于阶数 M ，则将 $[85, 91, 97]$ 分裂为两个结点 $[85, 91]$ 和结点 $[97]$ ，关键字 95 插入到结点 $[95, 97]$ 中，并将关键字 91 上移至其双亲结点中，发现其双亲结点 $[72, 97]$ 中包含的关键字的个数 2 小于阶数 M ，插入操作完成。



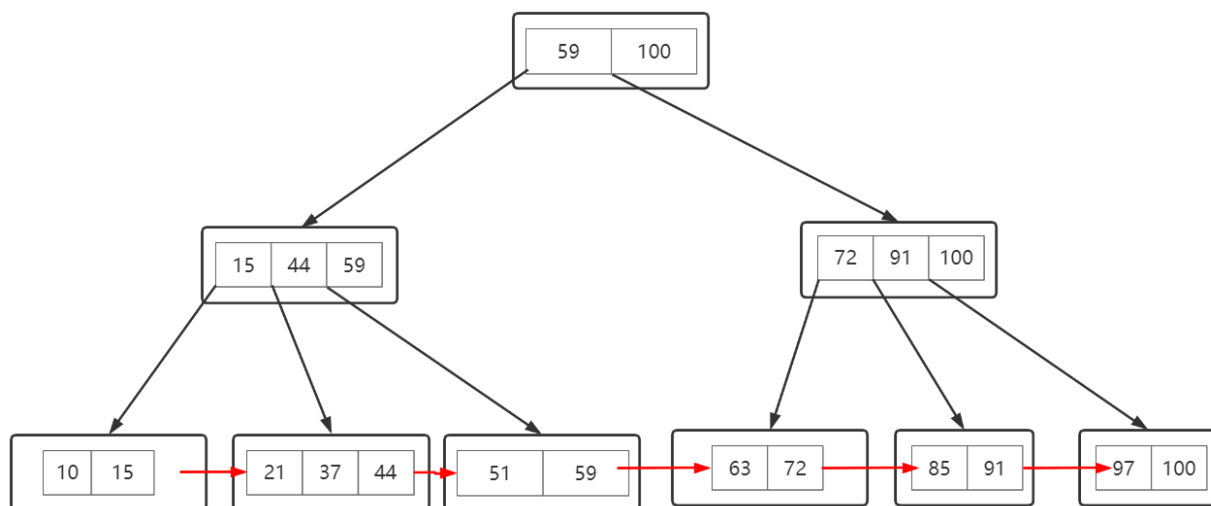
3、在第 2 情况中，如果上移操作导致其双亲结点中关键字个数大于 M ，则应继续分裂其双亲结点

插入关键字 40，按照第 2 种情况将结点分裂，并将关键字 37 上移到父结点，发现父结点 $[15, 37, 44, 59]$ 包含的关键字的个数大于 M ，所以将结点 $[15, 37, 44, 59]$ 分裂为两个结点 $[15, 37]$ 和结点 $[44, 59]$ ，并将关键字 37 上移到父结点中 $[37, 59, 97]$ 。父结点包含关键字个数没有超过 M ，插入结束。



4、若插入的关键字比当前结点中的最大值还大，破坏了B+树中从根结点到当前结点的所有索引值，此时需要及时修正后，再做其他操作。

插入关键字 100，由于其值比最大值 97 还大，插入之后，从根结点到该结点经过的所有结点中的所有值都要由 97 改为 100。改完之后再分裂操作。

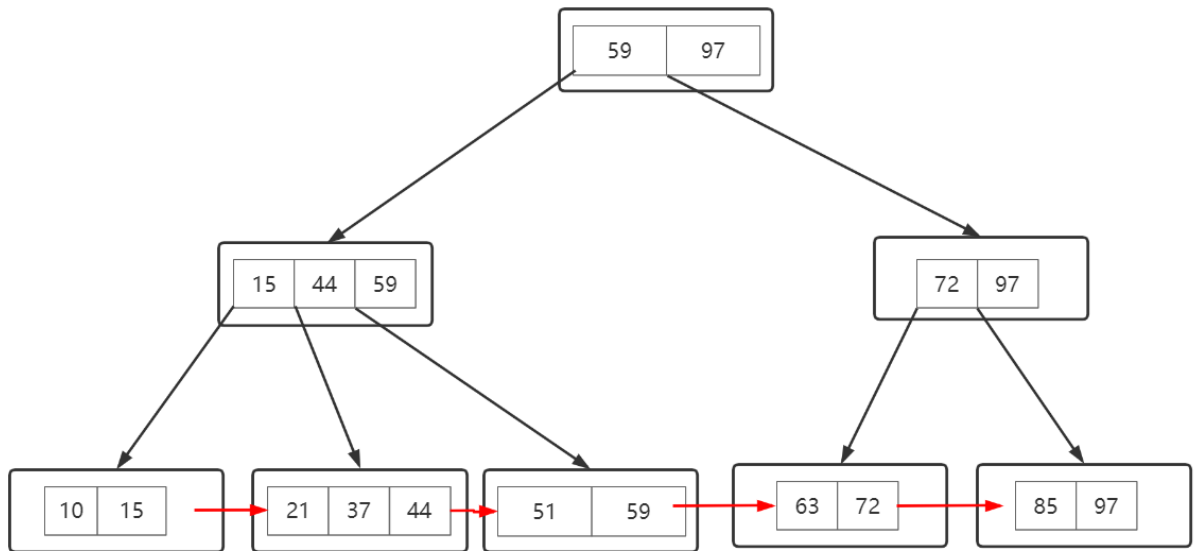


B+树的删除操作

“对于 B+ 的删除操作而言，与 B-树类似”，我想你笑了，那我们接着看，哈哈！在 B+ 树中删除关键字时，有以下几种情况：

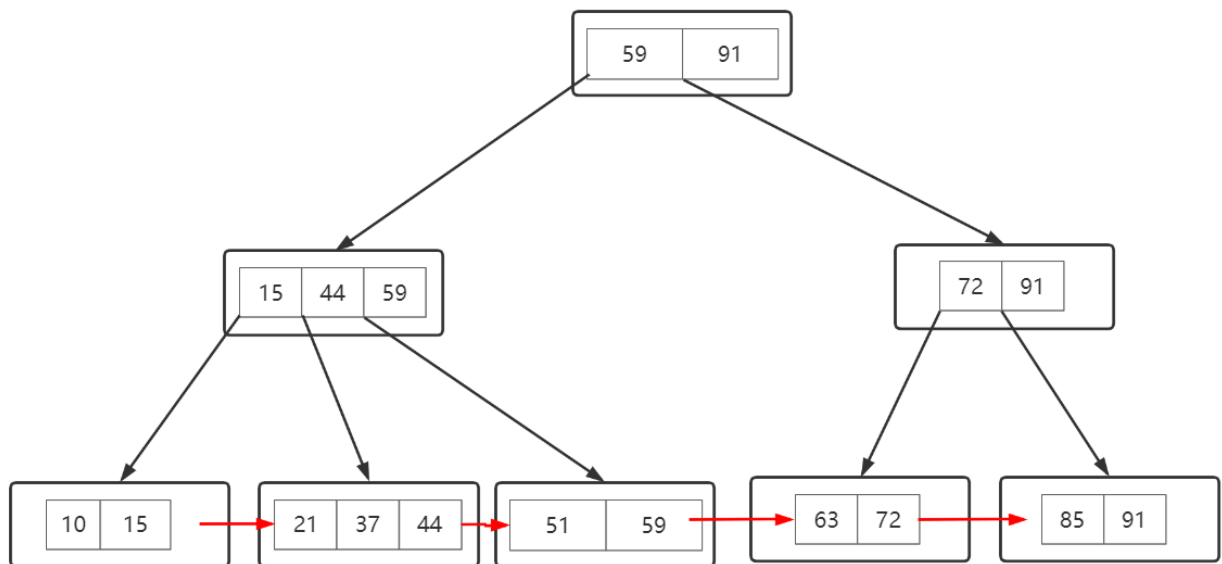
1、找到存储有该关键字所在的结点时，由于该结点中关键字个数大于 $\lceil M/2 \rceil$ ，做删除操作不会破坏 B+ 树，则可以直接删除。

删除关键字 91，包含关键字 91 的结点 [85、91、97] 中关键字的个数 3 大于 $\lceil M/2 \rceil = 2$ ，做删除操作不会破坏 B+ 树的特性，直接删除。



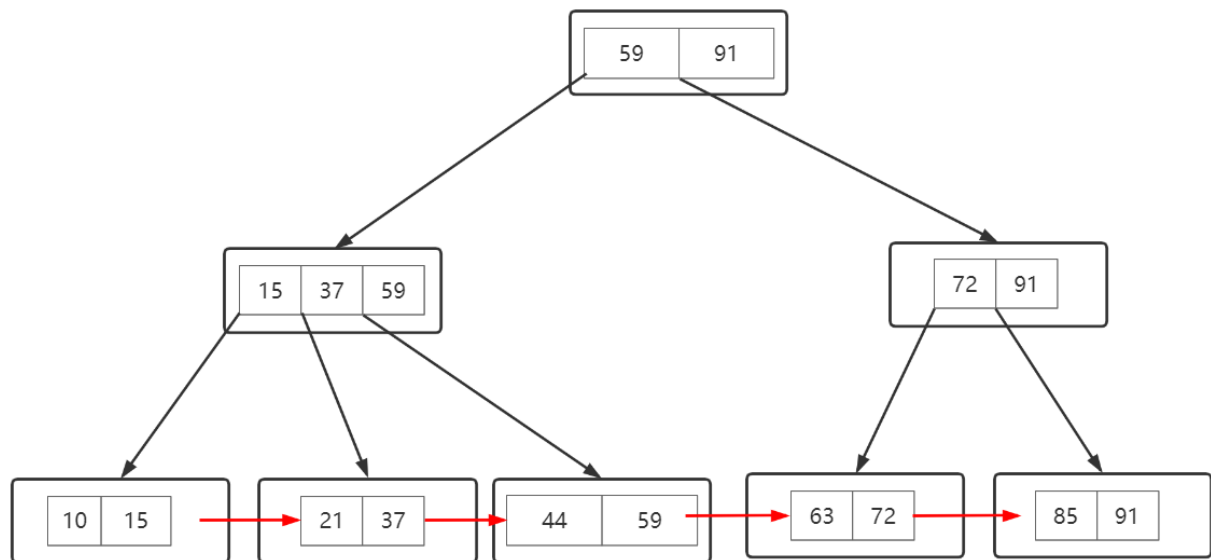
2、当删除某结点中最大或者最小的关键字，就会涉及到更改其双亲结点一直到根结点中所有索引值的更改。

以删除整颗 B+ 树中最大的关键字 97 为例，查找并删除关键字 97，然后向上回溯，将所有关键字 97 替换为次最大的关键字 91：



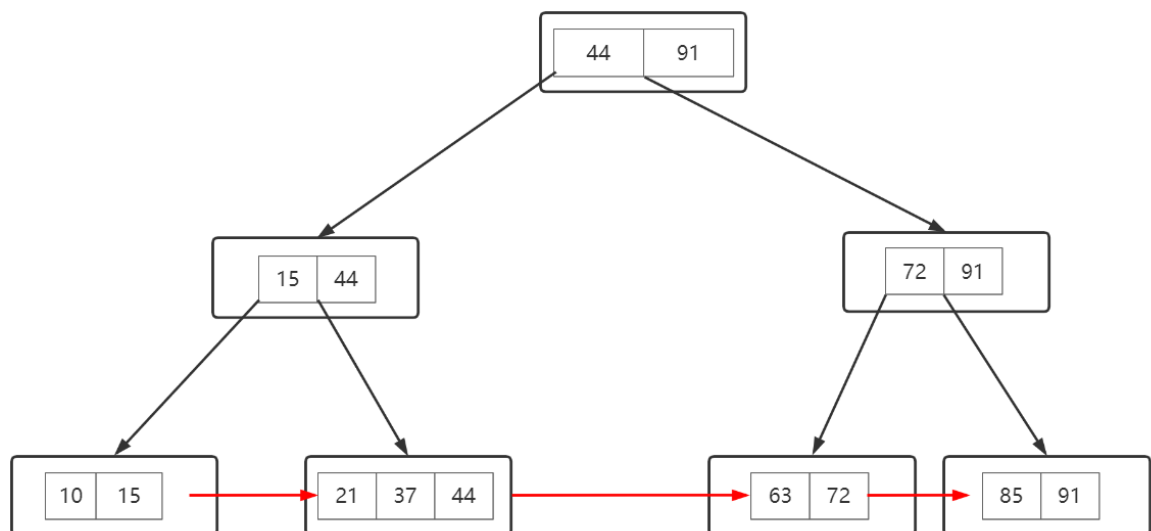
3、当删除该关键字，导致当前结点中关键字个数小于 $\lceil M/2 \rceil$ ，若其兄弟结点中含有多余的关键字，可以从兄弟结点中借关键字完成删除操作。

当删除某个关键字之后，结点中关键字个数小于 $\lceil M/2 \rceil$ ，则不符合 B+ 树的特性，则需要按照 3 和 4 两种情况分别处理。以删除关键字 51 为例，由于其兄弟结点 [21, 37, 44] 中含有 3 个关键字，所以可以选择借一个关键字 44，同时将双亲结点中的索引值 44 修改 37，删除以后如下图所示：



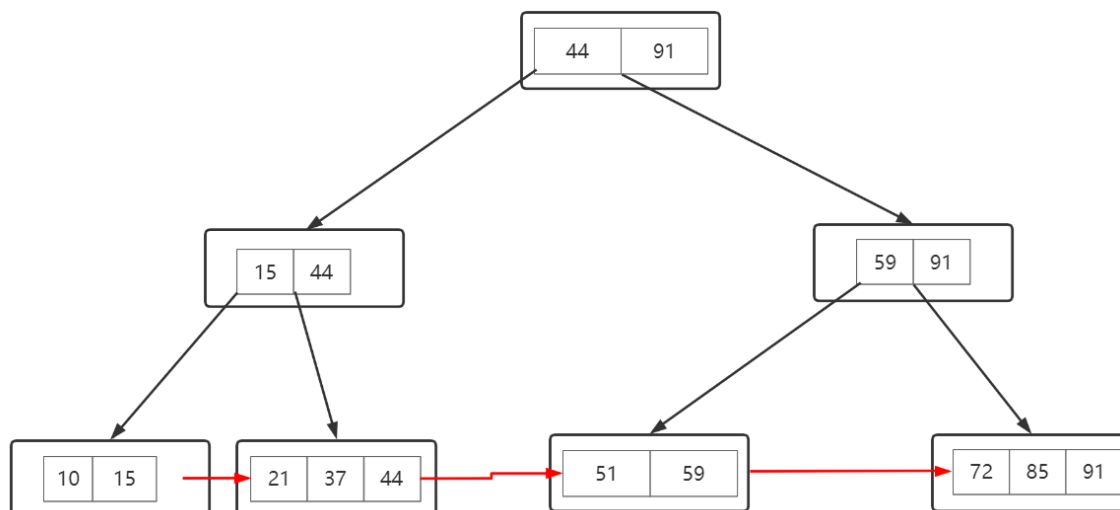
4、第3种情况中，如果其兄弟结点没有多余的关键字，则需要同其兄弟结点进行合并。为了说明这种情况，我们在第3种情况最终得到的B+树之上进行删除操作。

我们以删除上面这个B+树中的关键字59说明第4种情况，首先查找到关键字59所在结点[44, 59]，发现该结点的兄弟结点[21, 37]包含的关键字的个数2等于 $\lceil M/2 \rceil$ ，所以删除关键字59，并将结点[21, 37]和[44]进行合并[21, 37, 44]，然后向上回溯，将所有关键字59替换为次最大的关键字44：



5、当进行合并时，可能会产生因合并使其双亲结点破坏B+树的结构，需要依照以上规律处理其双亲结点。

删除关键字63，当删除关键字后，该结点中只剩关键字72，且其兄弟结点[85, 91]中只有2个关键字，所以将[72]和[85, 91]进行合并，向上回溯，删除结点[72, 91]当中的关键字72，此时结点中只有关键字91，不满足B+树中结点关键字个数要求，但其兄弟结点[15, 44, 59]中包含的3个关键字，所以从其兄弟结点当中借一个关键字59，再对其兄弟结点的父结点中的关键字进行调整，将关键字59替换为44。



总之，在 B+树中做删除关键字的操作，采取如下的步骤：

1. 删除该关键字，如果不破坏 B+树本身的性质，直接完成删除操作（情况 1）；
2. 如果删除操作导致其该结点中最大（或最小）值改变，则应相应改动其父结点中的索引值（情况 2）；
3. 在删除关键字后，如果导致其结点中关键字个数不足，有两种方法：一种是向兄弟结点去借，另外一种是同兄弟结点合并（情况 3、4 和 5）。（注意这两种方式有时需要更改其父结点中的索引值。）

B+树复杂度分析

B+树 是 B-树的一个升级版，在存储结构上的变化，由于磁盘页的大小限制，只能读取少量的B-树结点到内存中（因为B-树结点就带有数据，占用更多空间，所以说是 **少量**）；而B+树就不一样了。因为非叶子结点不带数据，能够一次性读取更多结点进去处理，所以对于同样的数据量，B+树更加 "矮胖"，性能更好。但是两者在查找、插入和删除等操作的时间复杂度的量级是一致的，均为 $O(\log n)$ 。

这个量级是如何得来的呢？我们一起来看看 1970 年计算机的先驱们是如何计算的。

首先假定 $h \geq 0$ 的整数（事实上就是树高）， k 是一个自然数（也就是B-树当中提到的最小度数）

一颗最小度为 k ，高度为 h 的 B-树 T ，或为空树，或者满足下面三个特性：

1. 从根结点到任意一个叶子结点有着相同的长度 h ，也称为树的高度。
2. 除根结点和叶子节点外，每一个内部结点至少有 $k+1$ 个孩子结点。根结点要么为叶子，要么至少包含两个孩子。
3. 每个结点至多有 $2k+1$ 个孩子结点。

设 N_{min} 和 N_{max} 分别表示 B-树中可以最少包含与最多包含的结点数目，则：

$$N_{min} = 1 + 2((k+1)^0 + (k+1)^1 + \dots + (k+1)^{h-2}) = 1 + \frac{2}{k}((k+1)^{h-1} - 1)$$

其中 $h \geq 2$ ，当 $h = 1$ 依然成立， N_{min} 至少包含一个结点，即根结点，之后每一层所包含的结点数目则为 $(k+1)^{h-2}$ ，其中 $k+1$ 表示每一个内部结点最少拥有的孩子结点数。类似的：

$$N_{max} = \sum_{i=0}^{h-1} (2k+1)^i = \frac{1}{2k}((2k+1)^h - 1); h \geq 1$$

则对于一颗 B-树所能包含的结点数目的上下限为：

$$N(T) = 0 \quad \text{if} \quad T \in \tau(k, 0);$$

$$1 + \frac{2}{k} \left((k+1)^{h-1} - 1 \right) \leq N(T) \leq \frac{1}{2k} \left((2k+1)^h - 1 \right) \quad \text{otherwise.}$$

则对于一颗 B-树所能包含的关键字的最大数目和最小数目分别为 和：

$$I_{\min} = 1 + k \left(2 \frac{(k+1)^{h-1} - 1}{k} \right) = 2(k+1)^{h-1} - 1$$

$$I_{\max} = 2k \left(\frac{(2k+1)^h - 1}{2k} \right) = (2k+1)^h - 1$$

则可以得到 B-树高度的一个上下限：

$$\log_{2k+1} (I+1) \leq h \leq 1 + \log_{k+1} \left(\frac{I+1}{2} \right) \quad \text{for} \quad I \geq 1,$$

$$h = 0 \quad \text{for} \quad I = 0.$$

这也就是我们所期待的对数级别的时间复杂度。对于上面的这个过程不清楚的可以跳过看下面。

对于一颗结束为m的B-树而言，每一个结点最多可以存储m个关键字，假设总共有 N 个关键字，树的高度为 h，且这颗树完全填满，则 $N = m^h$ ，继而得到 $h = \log N$ (m为底)，也就是树的高度为 $\log(N)$ (m为底)，可能有的人又会问，那一个结点中包含 m 个关键字，这m 个关键字只有进行顺序遍历才能知道接着选择哪一个孩子结点，需要O(m)的时间，你说的对也不对，对是因为一个结点内的关键字的遍历确实是需要O(m)的时间复杂度，但是这个遍历是在内存当中进行的，时间一般可以忽略。我们通常更加关注的是磁盘 I/O 的次数，也就是 $\log(N)$ (m为底)，所以 B-树或者 B+树的时间复杂度就是 $O(\log(N)$ (m为底))。